YaniMed | System Design v4

PLE Reviewer AI Search Engine | Revised October 2026

Purpose and design boundary

YaniMed is a single-user beta for reviewing Philippine Physician Licensure Examination (PLE) PDF materials. Users upload text-based or scanned reviewer PDFs, ask typed or dictated questions, and receive explanations grounded **only in uploaded documents**, with validated chapter and page citations. Voice playback is optional. The beta targets PHP 0/month where free-tier limits permit; this is not a guarantee of unlimited usage.

Updated system architecture

Layer	Design / responsibility
Frontend	Next.js + TypeScript (localhost:3001). Chat, upload, citation chips/PDF viewer, microphone, speaker, Settings.
Backend	NestJS + Prisma (localhost:3000). RAG, citation validation, PDF ingestion, voice proxy, access controls. Secrets remain server-side.
Generation gateway	OmniRoute (localhost:20128), combo **ple-answer**. Priority: **Gemini** → **Groq** → **OpenRouter**. Automatic fallback is a gateway concern, not a second retrieval source.
Embedding	Direct Gemini embeddings API, pinned model/output dimension 768; not routed through OmniRoute.
Database and files	Supabase Postgres + pgvector(768) + tsvector for metadata/chunks/search; Cloudflare R2 for original PDFs.
Voice	ElevenLabs TTS and STT via NestJS endpoints. Free-tier access, models, and voice IDs must be verified before activation.
Development assistants	OpenCode (implementation), Google Antigravity (UI/browser verification), Kiro (specifications/test plans). **Not** OmniRoute inference providers.

AI routing: runtime versus development

Runtime answer route: User question → NestJS retrieval → evidence chunks → OmniRoute **ple-answer** → Gemini (primary), then Groq, then OpenRouter on configured failures → NestJS citation validation → answer in chat.

Kiro is not the fourth fallback. It is a development tool, not an established general-purpose model endpoint for this architecture. Unofficial adapters should not be treated as production-ready or authorized without separate review. Antigravity also stays outside the application runtime.

Document ingestion and grounded answers — planned

• Upload a PDF through NestJS; validate it, store the original in R2, and create a Document record with processing status.

• Process asynchronously and resumably. Read PDF bookmarks first for chapters, then heading heuristics and optional manual corrections.

• Extract text per page with pdfjs-dist; use Tesseract.js OCR only for scanned or low-text pages.

• Chunk roughly 300–500 tokens while preserving document ID, chapter, physical page and printed page label.

• Embed with a single consistent Gemini model at 768 dimensions and save searchable chunks in Supabase.

• At query time, combine pgvector semantic search and tsvector keyword search using Reciprocal Rank Fusion (RRF).

• Generate only from retrieved context. Validate chapter/page citations against retrieved source chunks; abstain if evidence is missing.

• Render clickable citations that navigate to the corresponding page in the PDF viewer.

Voice interaction — planned

Feature	Expected behavior
Text-to-speech	Speaker button beneath an AI answer synthesizes that answer through NestJS/ElevenLabs. Play/stop, loading state, one active audio stream, cleanup on navigation; omit Markdown and citation markers from spoken text.
Speech-to-text	Live microphone button requests permission, records and releases mic tracks, sends audio to NestJS/ElevenLabs, then inserts editable transcript into the composer. Never auto-send.
Voice preferences	Persist selected verified voice ID, client-side playback speed, and recognition language. Begin with one confirmed accessible voice rather than relying on a free-tier Voice Library listing.
Other Settings	Appearance persistence already exists; General, Notifications, Storage, and Personalization require independent functional implementation and truthful limits.

Proposed backend API contracts

• POST /voice/synthesize — validated answer text, selected voice ID; returns audio with correct content type.

• POST /voice/transcribe — validated multipart audio; returns transcript text for user editing.

• GET /voice/voices — optional only if account permissions support it; otherwise use a vetted configured voice list.

• Require backend-only ELEVENLABS_API_KEY; apply development-only route guard until authentication, quotas, and rate limits are in place.

Security and operating constraints

• Do not expose Gemini, OmniRoute, R2, or ElevenLabs secrets in browser bundles, chat logs, or screenshots.

• Audio and spoken questions are sent to an external service; disclose this and avoid patient-identifying material.

• Enforce file size, duration, MIME type, text length, and rate limits. Release microphone tracks on stop, unmount, and errors.

• Verify provider model access, free-tier credits, Filipino/Taglish behavior, and commercial rights before making claims.

• Protect manually maintained pgvector/GIN indexes: do not run prisma migrate dev against the existing database.

Implementation ownership and change control

Tool	Role and boundary
OpenCode	Primary engineer: backend, PDF ingestion, retrieval, ElevenLabs integration, and integration tests.
Google Antigravity	Frontend inspection, browser-based QA, and visual regressions; propose changes via isolated branch.
Kiro	Requirements, acceptance criteria, architectural review, and test specifications; not a runtime inference provider.
ChatGPT	Architecture review, task breakdown, verification checklists, and documentation.

Coordination rule: Do not let multiple agents edit the same working tree concurrently. Use Git branches or worktrees, review diffs, and merge tested increments. No agent should receive unnecessary production secrets or permission for destructive database operations.

Current status — verified versus planned

Area	Status
OmniRoute / LLM	Working locally: ple-answer combo and provider requests previously returned HTTP 200. Gemini was observed serving combo traffic; forced fallback behavior still needs end-to-end testing.
Supabase / Prisma	Connected; seven tables and existing vector(768)/tsvector indexes verified. No PDFs ingested at last check.
NestJS	Local API and AI health/test endpoints returned HTTP 200.
Next.js chat	Frontend runs; live microphone button lacked a handler; existing speaker message component was orphaned at last inspection. Phase 0 integration needed.
PDF-grounded RAG	Ingestion, hybrid retrieval, verified citations and end-to-end chat integration not yet implemented.
ElevenLabs	Free account established; credential setup and API access validation in progress. TTS/STT not yet confirmed working.
Settings	Appearance preference persists; other settings and voice controls largely placeholders.

Revised build sequence

- **Phase 0:** Render actual chat messages, make composer controlled, preserve existing UI; no fake answers/citations.
- **Phase 1:** Add secure development-only NestJS voice endpoints and validate ElevenLabs model/key access.
- **Phase 2:** Implement speaker TTS with stop/loading/single-player cleanup.
- **Phase 3:** Implement microphone STT with editable transcript and permission/error handling.
- **Phase 4:** Persist verified voice settings; leave unrelated Settings pages untouched until scoped.
- **Phase 5:** Implement resumable PDF ingestion, embeddings, hybrid retrieval, and citation validation.
- **Phase 6:** Wire real chat API, PDF viewer/citations, then end-to-end tests and production hardening.

Document note: this revision updates the supplied v2/v3 project summaries and the subsequent architecture discussion. Future provider quotas, model availability and pricing require revalidation.